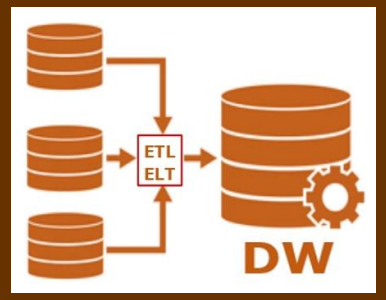
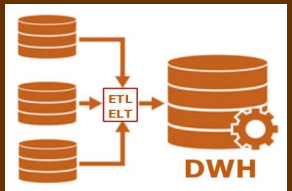


Data Warehousing

Follow us on :

YouTube Linkedin

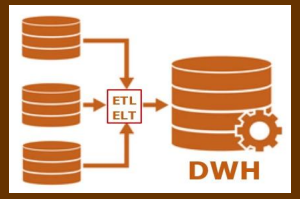
CDU
Cloud And Data Universe



What is a Data warehouse?

- Data warehouse is a centralized repository or place where data is stored.
- Abbreviations used are DW or DWH.
- Also known as Enterprise data warehouse (EDW)
- Data is loaded from one or more sources.
- DW stores historical and current data

Cloud And Data Universe

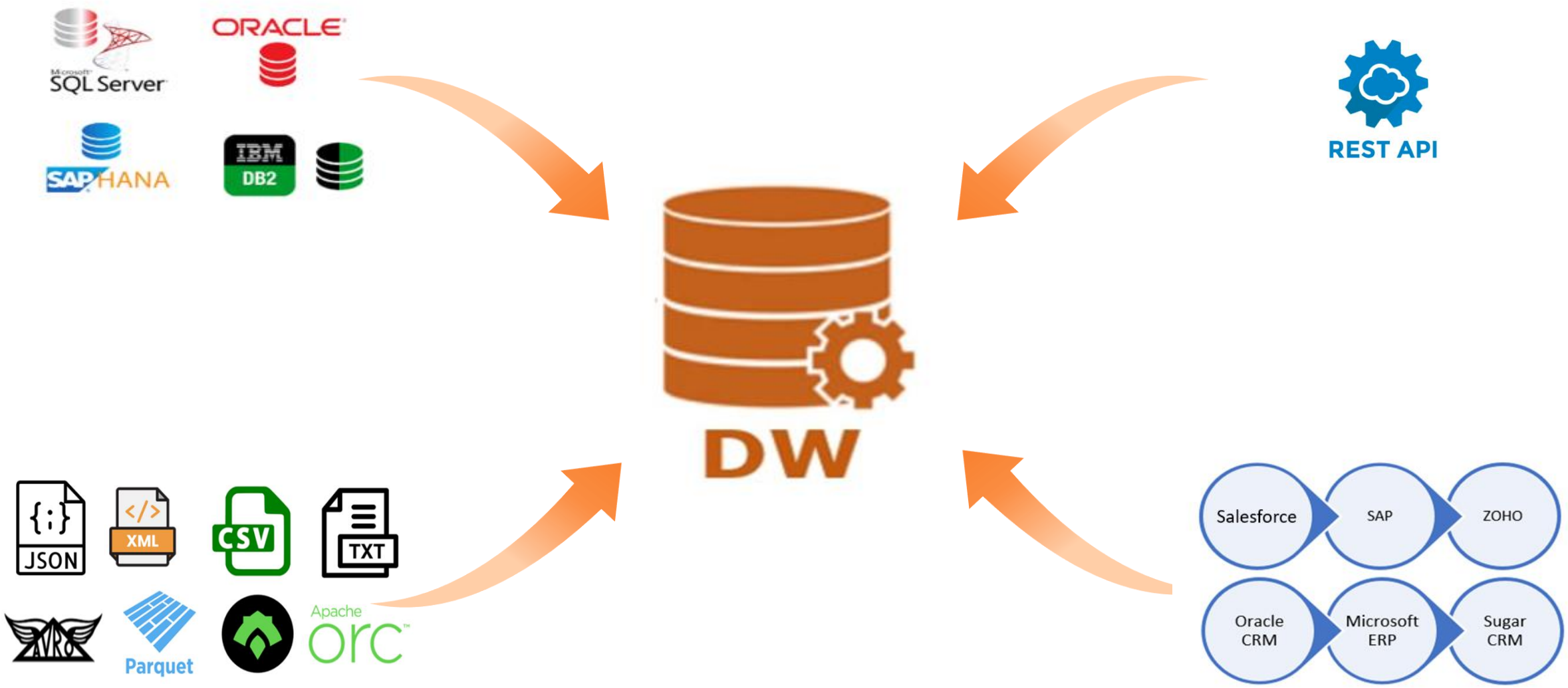


ETL/ELT

- ETL(Extract - Transform Load) / ELT(Extract - Load - Transform)
- Data load is performed by ETL process using different tools like SSIS, ADF, Informatica, etc.

Cloud And Data Universe

Data Warehouse Model



ETL/ELT Tools

Cloud And Data Universe

Azure Data Factory

Microsoft SQL Server Integration Services

HITACHI Pentaho

talend Talend Open Studio

Stitch Stitch

Informatica Informatica PowerCenter

ORACLE Oracle Data Integrator

Fivetran Fivetran

Airbyte Airbyte

SINGER Singer

integrate.io Xplenty (Integrate.io)

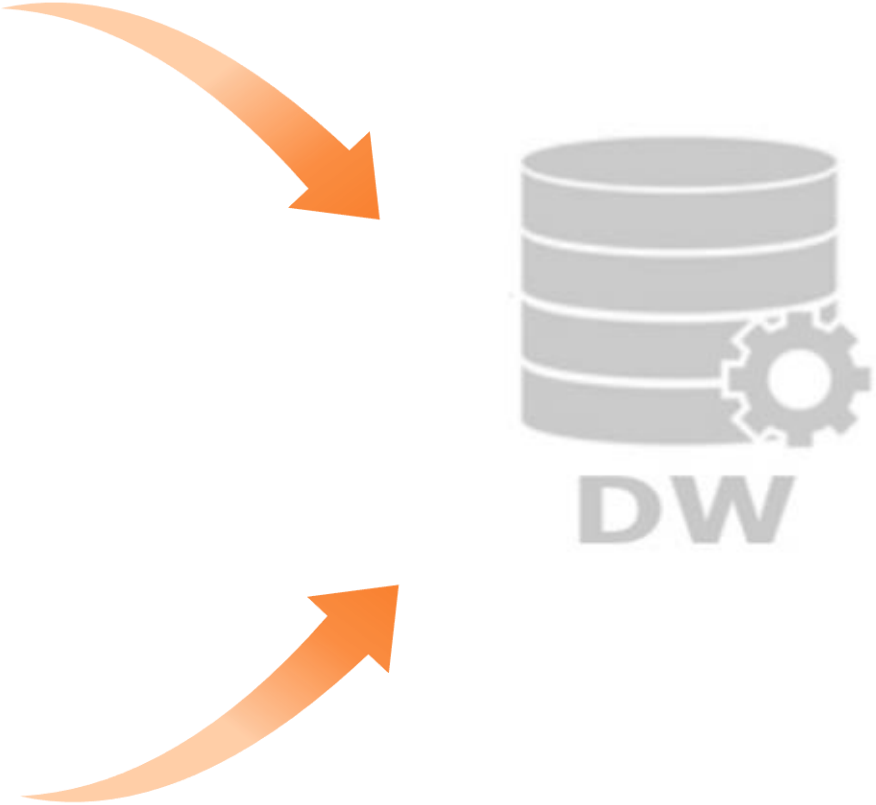
Dataddo

StreamSets

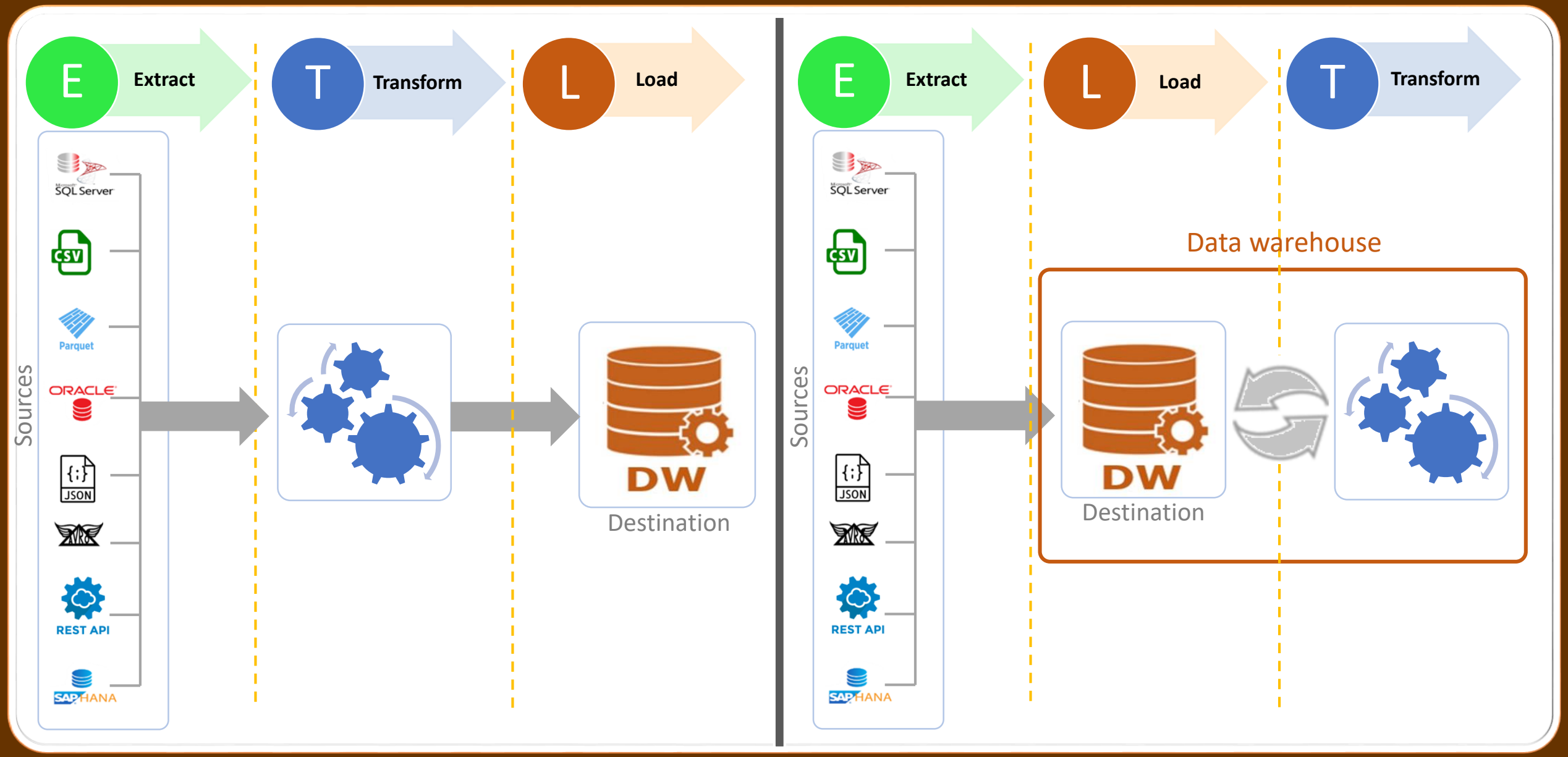
BLENDO

IBM DataStage

aws Glue



ETL/ELT Process



DW Tools

Cloud And Data Universe

Azure Data Factory

Microsoft SQL Server Integration Services

HITACHI Pentaho

talend Talend Open Studio

Stitch Stitch

Informatica Informatica PowerCenter

ORACLE Oracle Data Integrator

Fivetran Fivetran

Airbyte Airbyte

SINGER Singer

integrate.io Xplenty (Integrate.io)

Dataddo

StreamSets

BLENDO

IBM DataStage

aws Glue

amazon REDSHIFT

PostgreSQL

Azure Synapse Analytics

IBM DB2

IBM Cloud Db2 Warehouse on Cloud

DW

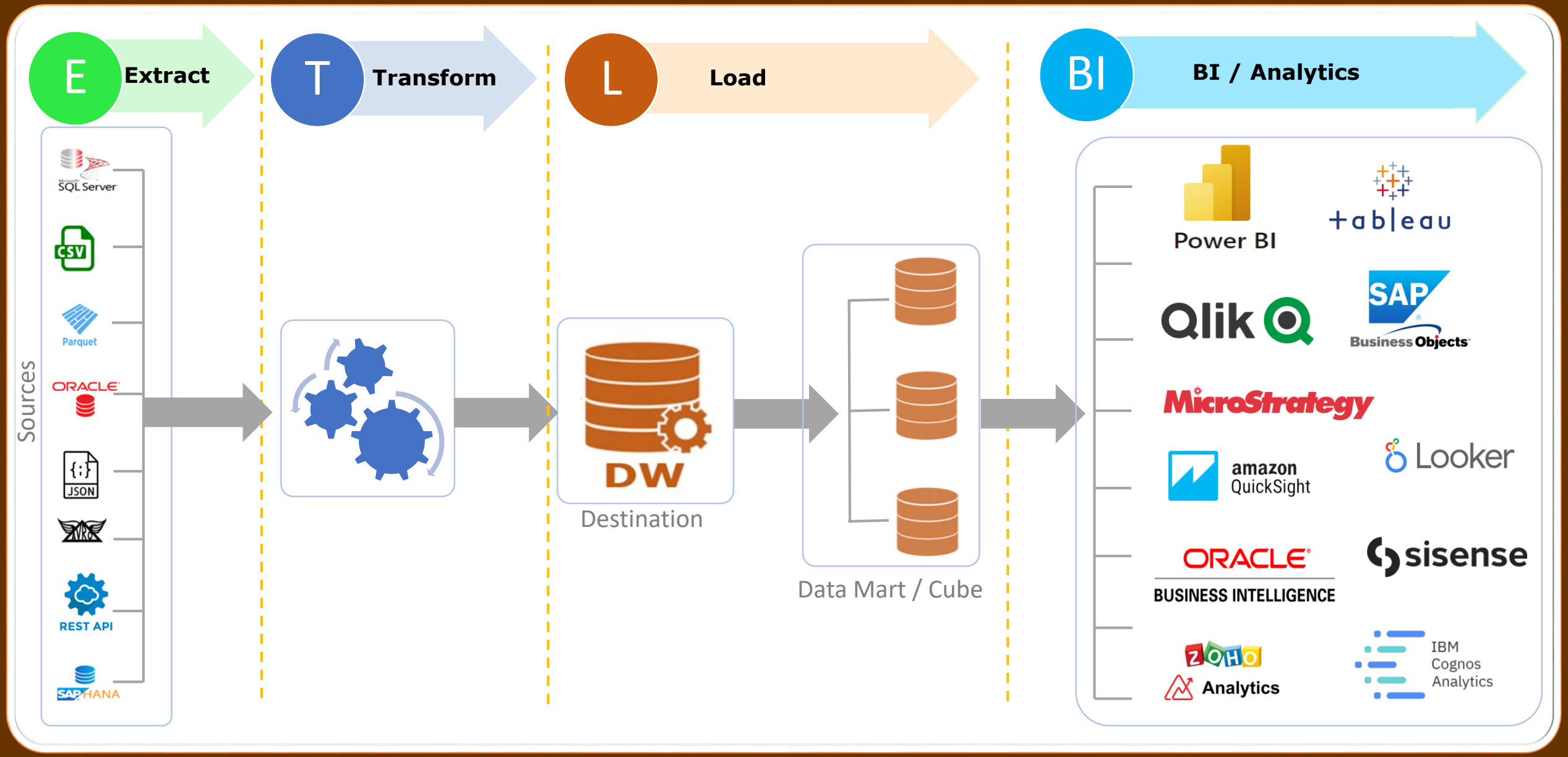
ORACLE DATA WAREHOUSE

snowflake

TERADATA

Google Big Query

Datawarehouse & Analytics Architecture



Data Model

- Data Model typically consists of multiple tables.
- These tables are called Fact and Dimensions.
- These tables are related to each other by relationship between them.
- Mostly you will see more Dimension tables and fewer Fact tables.

Dimension Tables

- Dimension tables are mostly designed per entity wise. For instance, you will see one dimension table each for products, customers, suppliers, date, vendors, employees, etc.
- In these tables you will have a primary key which acts as record identifier.
- Dimension tables can also be called as master tables.

When will be a new row/record added in dimension table?

- Consider an Employees table: A new record will only be added when a new employee joins an organization.
- Same is the case with other dimension tables.
- Hence, you will see data in Dimension tables will not be appended/inserted frequently as compared to Fact tables

Fact Tables

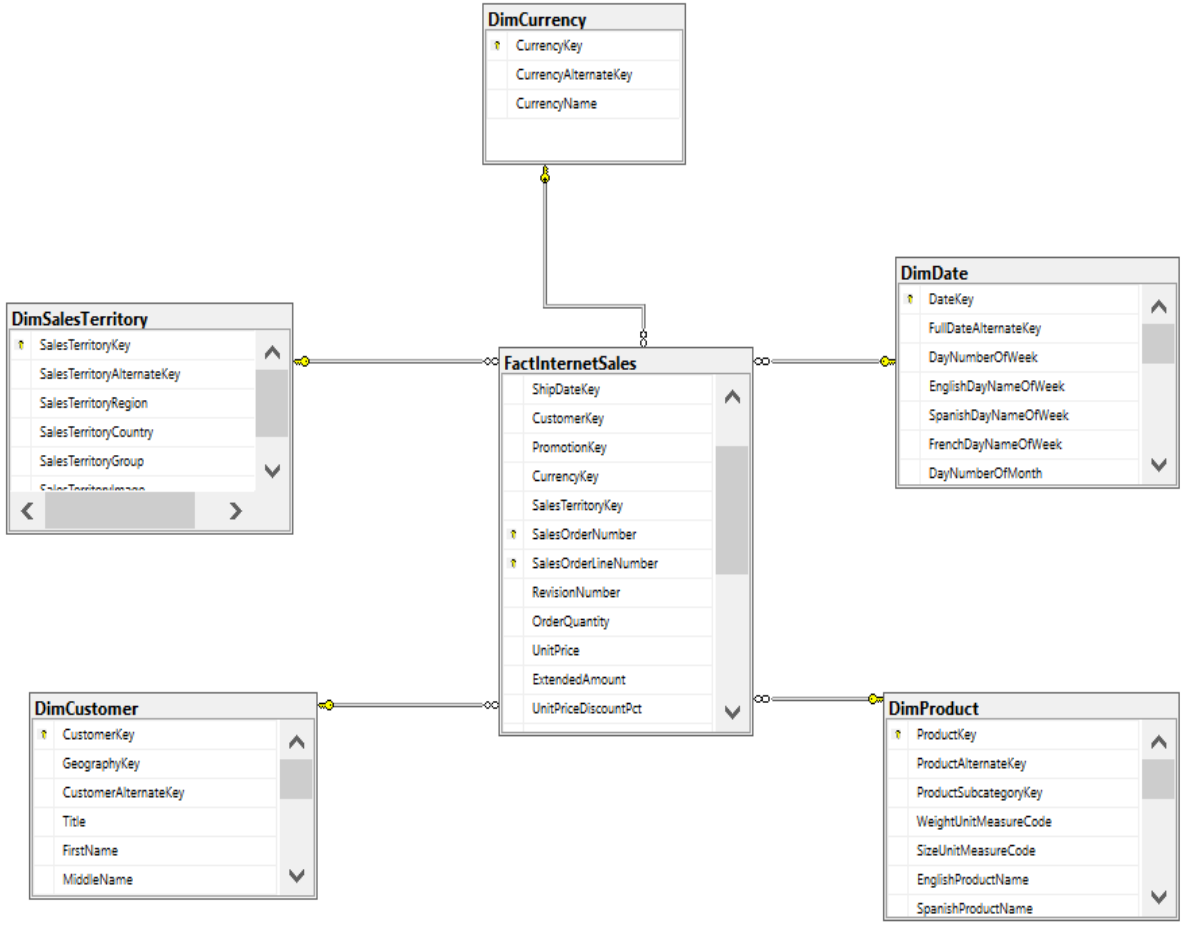
- Fact tables are transaction tables.
- Consider a sales tables: A new row/record will be inserted for every single sales transaction. Hence, data will be appended/inserted very often as compared to Dimension table.

Fact Tables

- Fact tables contain foreign key columns which are coupled with primary key columns in Dimension tables to maintain referential integrity.

Star Schema

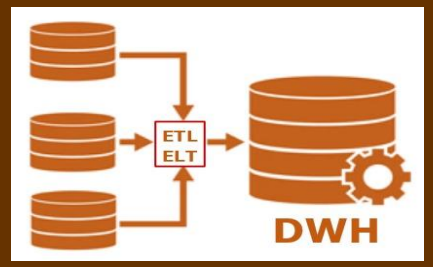
Star schema is a data model in which Dimension tables are directly connected to Fact table



Snowflake Schema

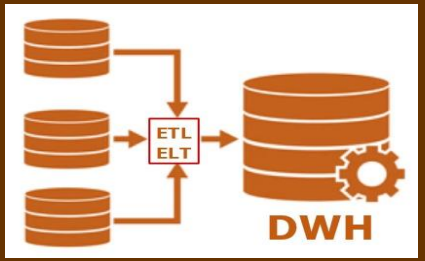
Snowflake schema is a data model in which Dimension tables may or may not be directly connected to Fact table

OLTP & OLAP



- OLTP stands for Online Transaction Processing
- OLAP stands for Online Analytical Processing

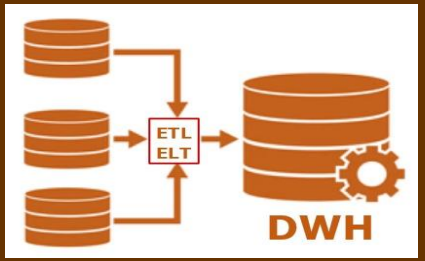
Differences between OLTP & OLAP



Period

OLTP	OLAP
Deals with most recent data	Deals with historic data

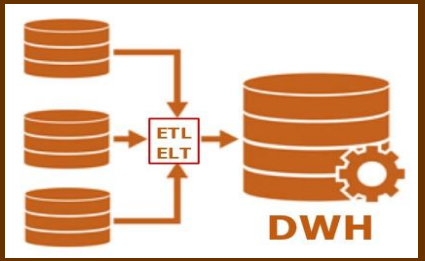
Differences between OLTP & OLAP



Processing

OLTP	OLAP
Processes small to large number of small transactions	Processes massive volume of data

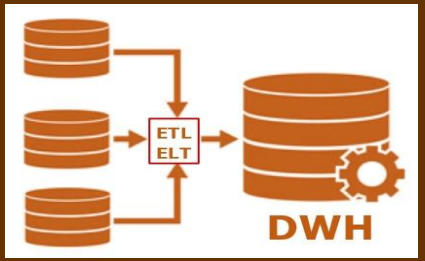
Differences between OLTP & OLAP



Queries

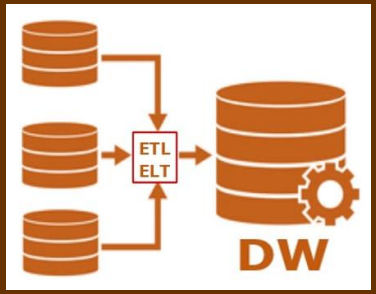
OLTP	OLAP
Simple queries are used	Complex queries are used
Uses INSERT, UPDATE & DELETE	Mostly SELECT only

Differences between OLTP & OLAP



Learning

OLTP	OLAP
Easy to learn	Need to invest time, doesn't come quickly



2. DATAWAREHOUSING

DATA LOADS

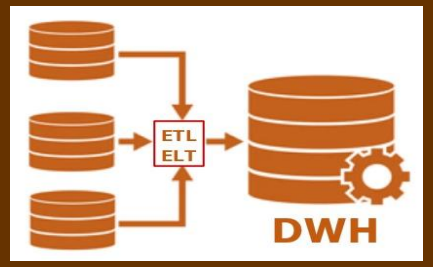
Follow us on :

YouTube

LinkedIn

CDU

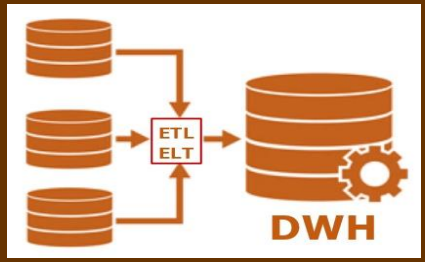
Cloud And Data Universe



1. Full load

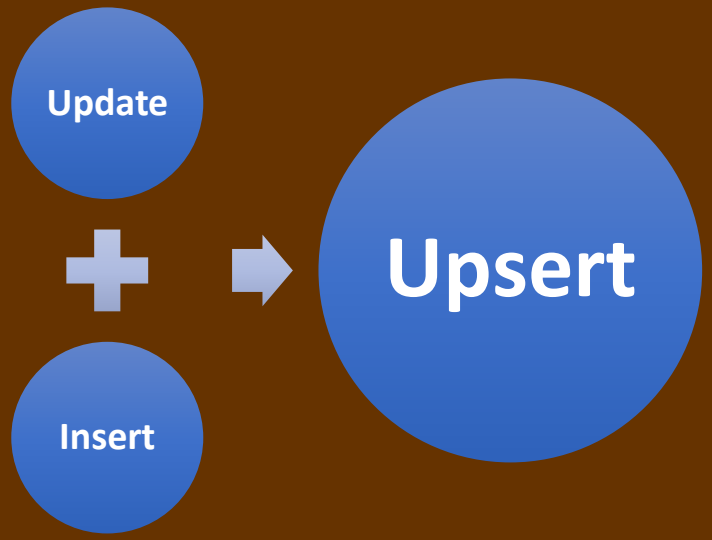
- Truncate & Load

Cloud And Data Universe

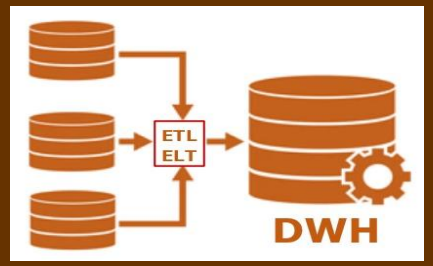


2. Upsert

- Combination of Update & Insert

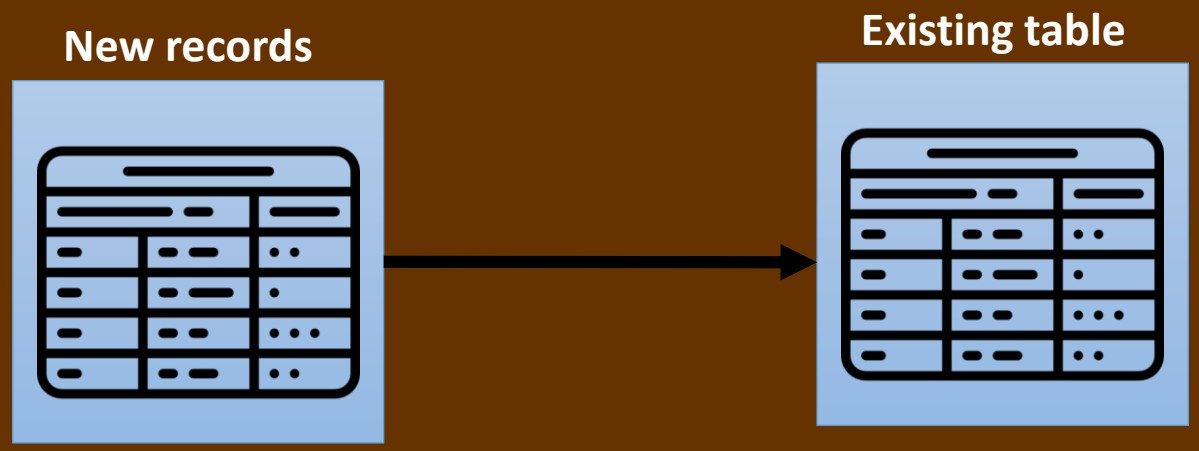


Cloud And Data Universe

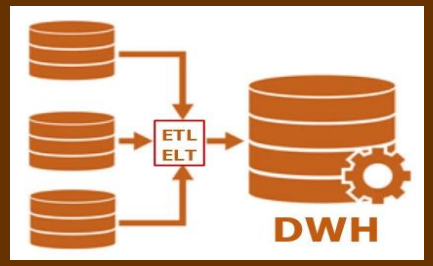


3. Incremental Load

- Identify & load new (delta) records mostly based on datetime

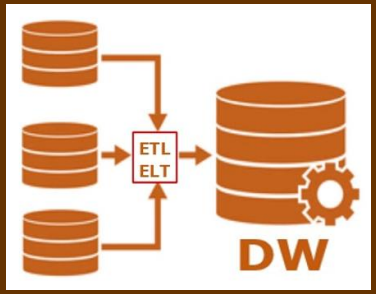


Cloud And Data Universe



4. Slowly changing dimensions (SCD)

- SCD1 - Update/overwrite changes
- SCD2 - Addition of new row
- SCD3 - Addition of new column



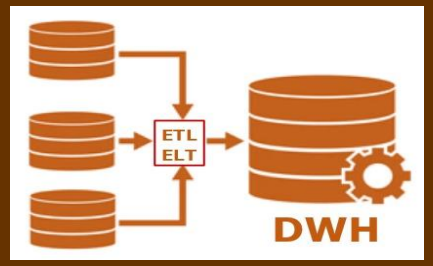
3. DATAWAREHOUSING

STORAGE LAYOUT MODELS

Follow us on :

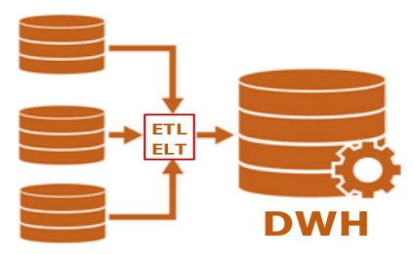
YouTube Linkedin

CDU
Cloud And Data Universe



STORAGE LAYOUT MODELS

1. ROW-STORE
2. COLUMN-STORE
3. HYRBID-STORE



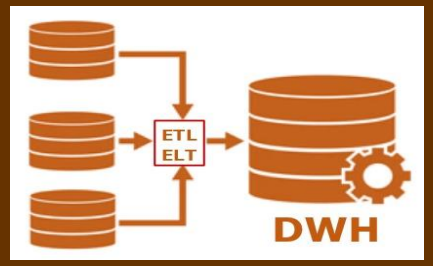
ROW-STORE

LOGICAL

EMPID	EMPNAME	SALARY
1	JOHN	4551
2	AMANDA	1105
3	SAM	4790
4	RAKESH	3720
5	AMAN	1223
6	RAHUL	3788

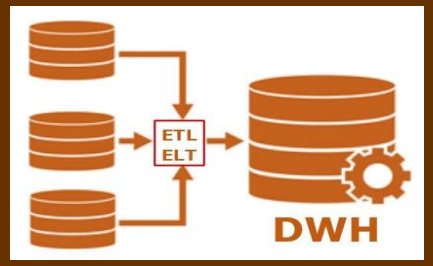
PHYSICAL

1	JOHN	4551	2	AMANDA	1105	3	SAM	4790
4	RAKESH	3720	5	AMAN	1223	6	RAHUL	3788



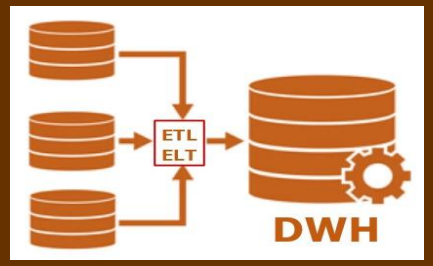
ROW-STORE

- Row-store / Row-Wise storage is horizontal partitioning.
- This is suitable when you need to insert or update a record.
- This model affects entire row where it scans through all the columns



ROW-STORE

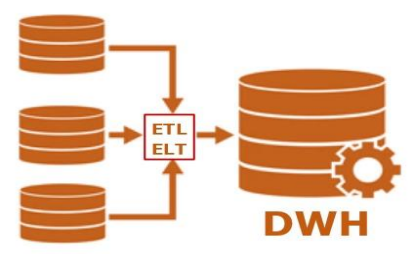
- Slower reads.
- Not optimized for querying.
- Used in case of frequent transactions.
- No efficient compression.



ROW-STORE

- Consider we need to sum up the salary column.
- In row store model it will scan through all the rows which is not performant.

EMPID	EMPNAME	SALARY
1	JOHN	4551
2	AMANDA	1105
3	SAM	4790
4	RAKESH	3720
5	AMAN	1223
6	RAHUL	3788



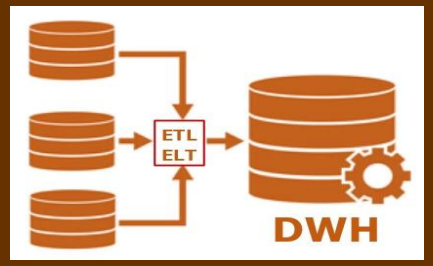
COLUMN-STORE

LOGICAL

EMPID	EMPNAME	SALARY
1	JOHN	4551
2	AMANDA	1105
3	SAM	4790
4	RAKESH	3720
5	AMAN	1223
6	RAHUL	3788

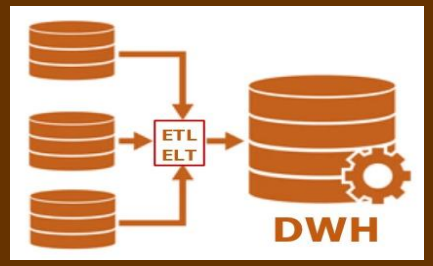
PHYSICAL

1	2	3	4	5	6	JOHN	AMANDA	SAM
RAKESH	AMAN	RAHUL	4551	1105	4790	3720	1223	3788



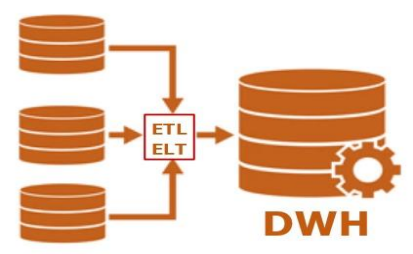
COLUMN-STORE

- Column-store model is vertical partitioning.
- In this model values of same column are stored continuously.
- In case where we need to extract specific column this model is preferred as it can easily extract those column/s without scanning entire data, which makes it extremely performant.



COLUMN-STORE

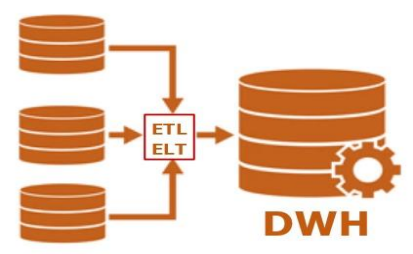
- Faster reads.
- Optimized for querying.
- Can’t be used in case of frequent transactions.
- Highly efficient compression.



COLUMN-STORE

- Consider we need to sum up up salary column.
- In row store model it will scan through all the rows which is not performant.

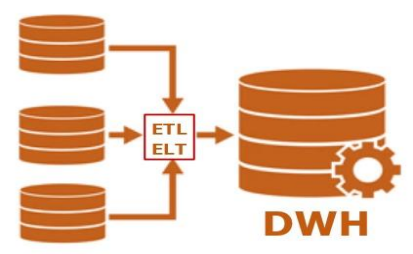
EMPID	EMPNAME	SALARY
1	JOHN	4551
2	AMANDA	1105
3	SAM	4790
4	RAKESH	3720
5	AMAN	1223
6	RAHUL	3788



COLUMN-STORE

1	2	3	4	5	6	JOHN	AMANDA	SAM
RAKESH	AMAN	RAHUL	4551	1105	4790	3720	1223	3788

- Consider we need to extract EMPNAME column.
- In this model it will easily extract the names as they are stored continuously, hence reducing the bottleneck vs row-store model.



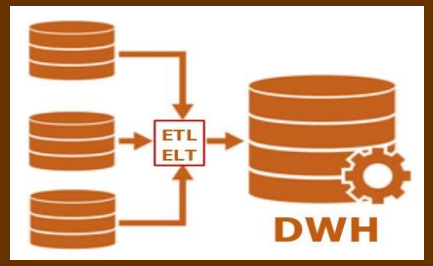
HYBRID-STORE

LOGICAL

EMPID	EMPNAME	SALARY
1	JOHN	4551
2	AMANDA	1105
3	SAM	4790
4	RAKESH	3720
5	AMAN	1223
6	RAHUL	3788

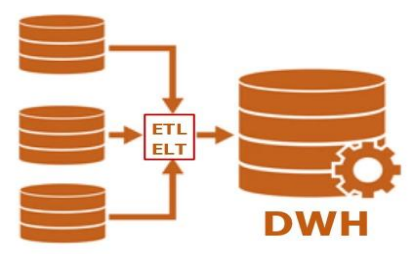
PHYSICAL

1	2	3	JOHN	AMANDA	SAM	4551	1105	4790
4	5	6	RAKESH	AMAN	RAHUL	3720	1223	3788



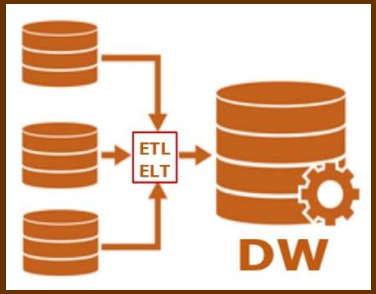
HYBRID-STORE

- Hybrid-store model combines both horizontal and vertical partitioning.



USE-CASES

- All these storage models are used in different SQL databases and file formats.



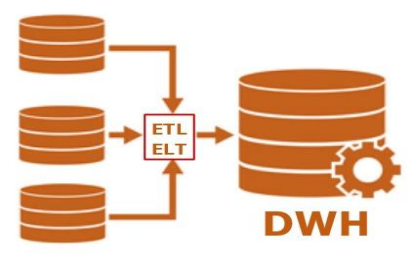
4.DATAWAREHOUSING

ROBIN ROUND DISTRIBUTION

Follow us on :

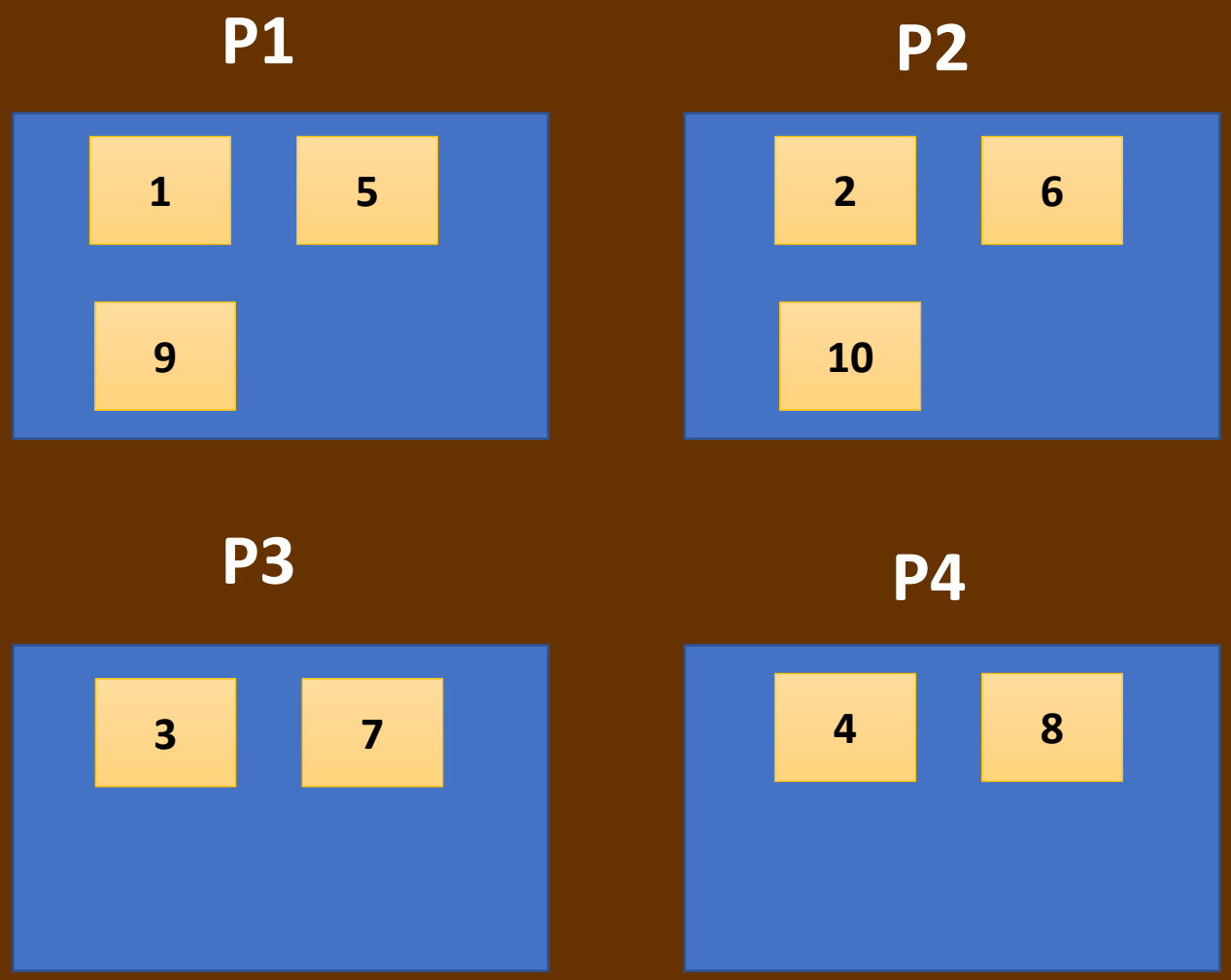
YouTube Linkedin

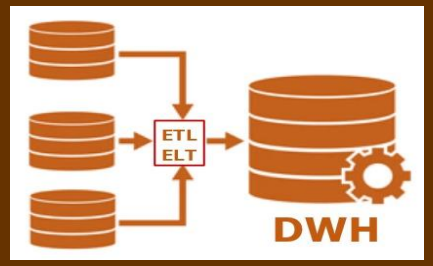
CDU
Cloud And Data Universe



Robin round distribution

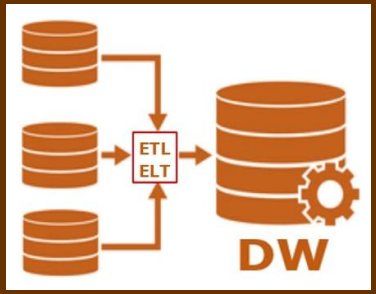
ID	QTY	PRICE	AMOUNT
1	10	10	100
2	5	3	15
3	1	2	2
4	20	10	200
..	..	..	..
..	..	..	..
10	100	3	300





Important points

- Robin round distributes data evenly across partitions in a sequential way.
- It is fastest way to load a data in table/partition.
- Useful in case of staging.
- Not performant when using joins as data needs to be shuffled.



5. DATAWAREHOUSING

HASH DISTRIBUTION

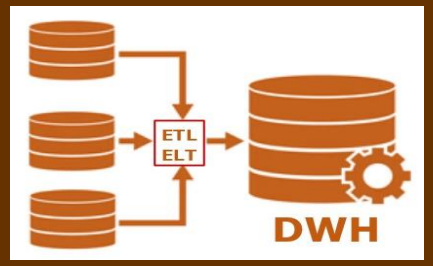
Follow us on :

YouTube

LinkedIn

CDU

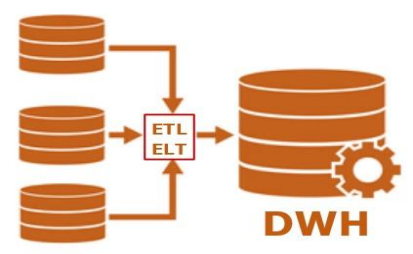
Cloud And Data Universe



HASH DISTRIBUTION

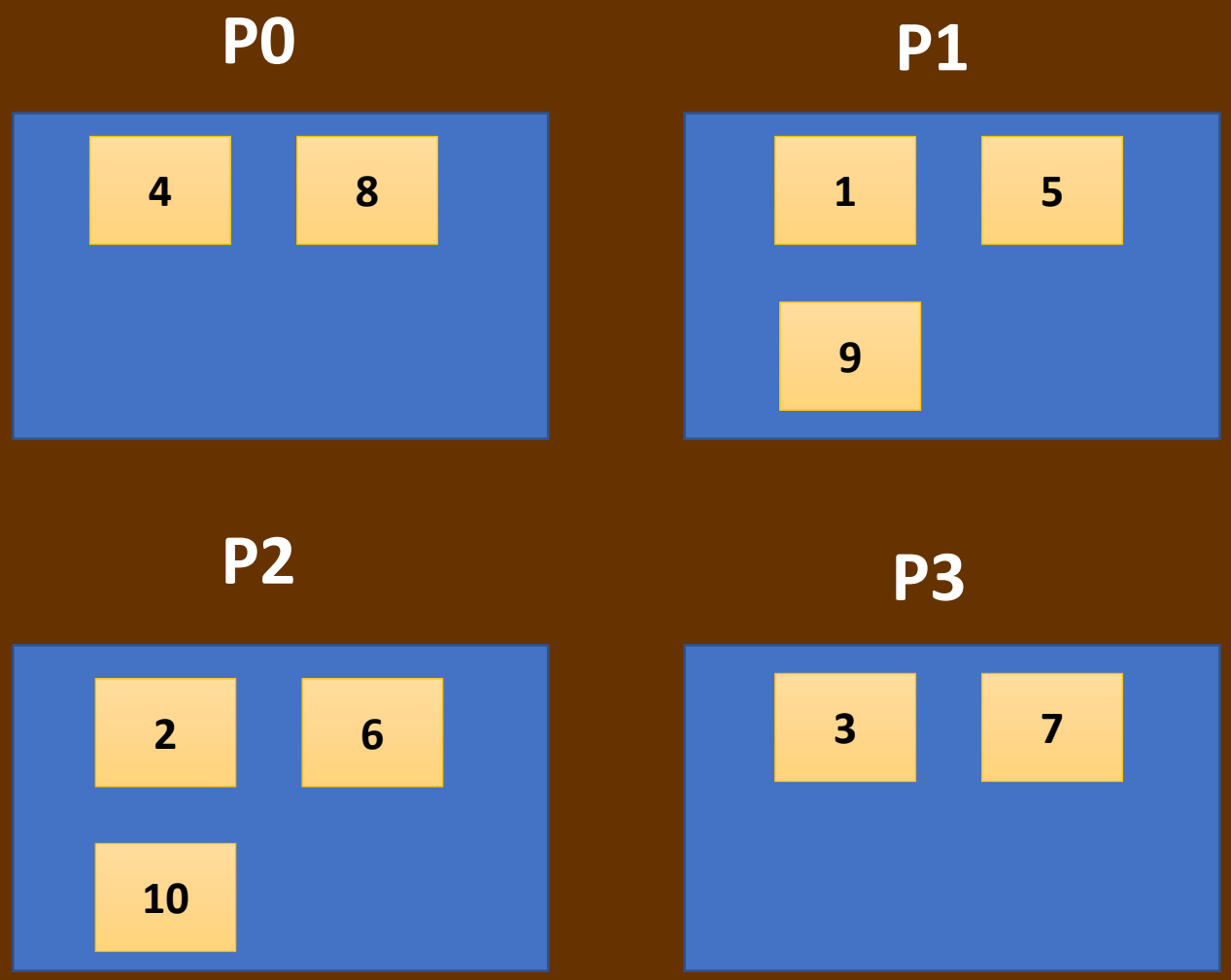
- Hash distribution uses a hash function to decide to pre determine which partition the row/value is to be assigned.

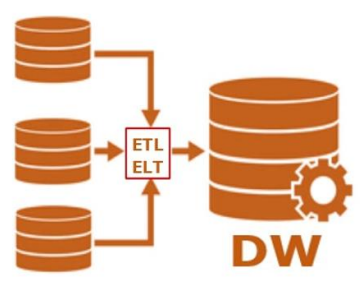
- Hash function returns a value called as hash value which is derived by using a calculation.



HASH DISTRIBUTION

ID	QTY	PRICE	AMOUNT
1	10	10	100
2	5	3	15
3	1	2	2
4	20	10	200
..	..	..	..
..	..	..	..
10	100	3	300





6. DATAWAREHOUSING

ACID PROPERTIES

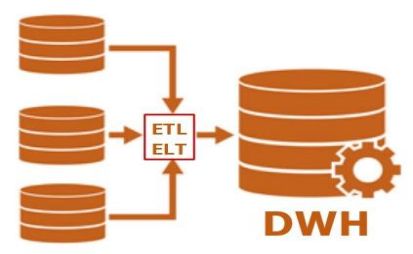
Follow us on :

YouTube

LinkedIn

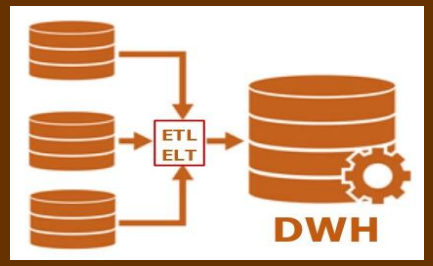
CDU

Cloud And Data Universe



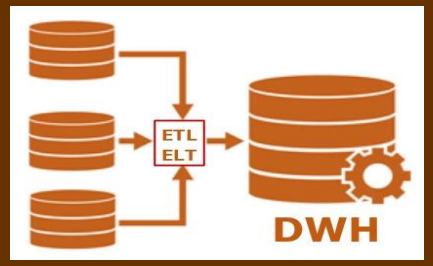
ACID Properties

- Atomicity
- Consistency
- Isolation
- Durability



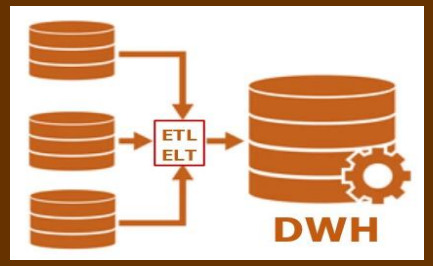
Transaction

- In databases and data storage systems, a transaction in any operation which can be treated as single unit of work.
- A transaction either completes fully or doesn’t complete at all, there can be only these 2 possibilities, it can’t have any other state in between.



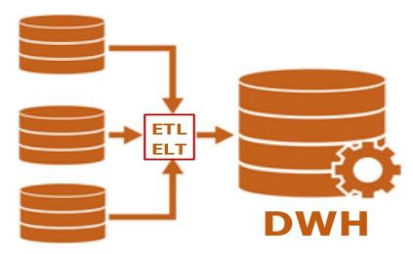
Transaction Example

- Consider you transfer money from one account to another.
- When this transaction occurs the amount is debited from one account and credited to another, it can’t be the case that it has debited from one account and not credited to other, this leads to imbalance.



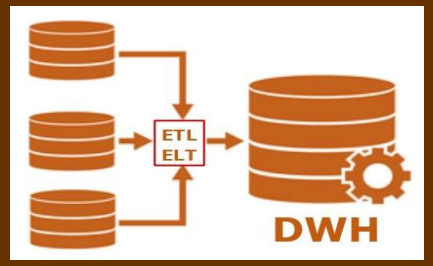
Atomicity

- In cases when you write, update or delete data it is executed as a transaction.
- This transaction is treated as single unit which can either execute completely or leave the state unchanged.
- This is important to prevent data loss and corrupting it.



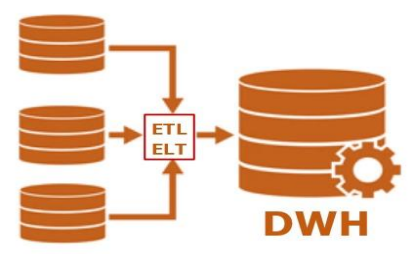
Atomicity

- When unwanted situations like system failure, network issues or other arise it can lead to a problem when transaction is running, this could lead to incomplete execution.
- Atomicity ensures either the transaction is successful or in case of any failures it rollbacks the transaction, leaving the state intact as it was earlier.



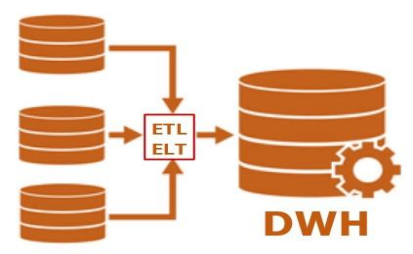
Atomicity

- Think about a common scenario we all would have experienced at least once while transferring money to other account, in case the transfer fails, the transaction is rolled back and money is credited back to the account from which transfer was done.



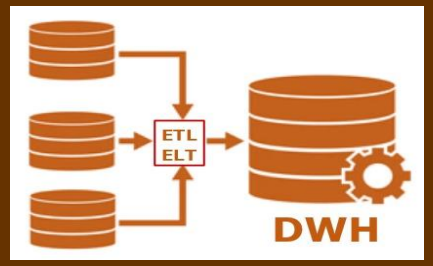
Consistency

- A transaction should ensure consistency of data is maintained.
- It is to ensure no violations of business rules or conditions(constraints) are done.
- Classic example, you can withdraw only the amount available in your bank account, if you try to withdraw more, the account can go into negative balance which will not be allowed.



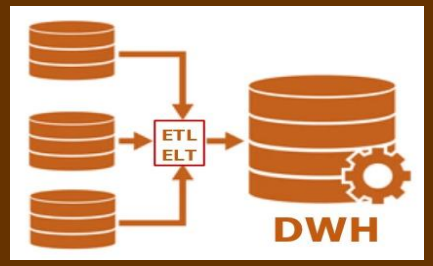
Consistency

- Similarly, business can define set of rules to be applied in a database to ensure they are met and not compromised leading to any issues which can have severe consequences.



Isolation

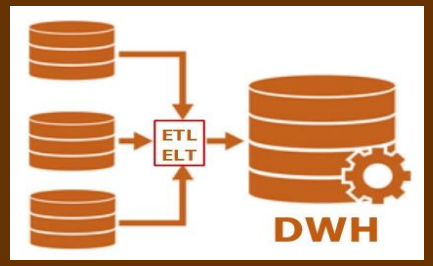
- Multiple users are performing read and write operations on a same table at same time, this is called as concurrency.
- The isolation property makes sure every single transactions runs individually in isolation and doesn’t interfere with other transactions.



Durability

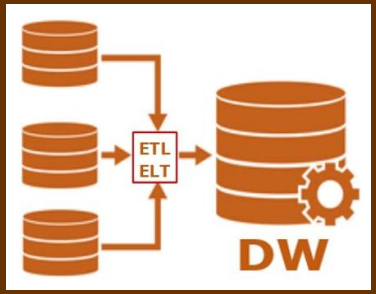
- Durability ensures when transactions are executed the changes committed will be saved successfully.

- This means once a transaction is committed it will remain committed forever and that's a guarantee!
- If that wasn't the case the data won't be trustworthy



Use cases

- Currently we can see ACID properties being implemented in various systems like databases and delta lake making it robust!



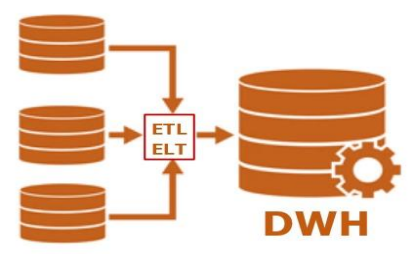
7. DATAWAREHOUSING

NORMALIZATION

Follow us on :

YouTube LinkedIn

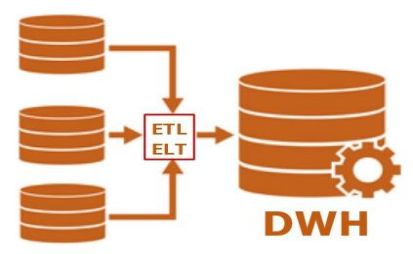
CDU
Cloud And Data Universe



Normalization

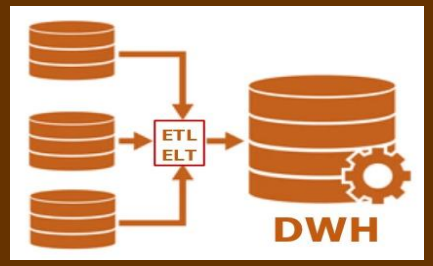
- Normalization in SQL is the process of organizing data in a database to reduce redundancy and improve data integrity.

- This process includes dividing the database or splitting the table into multiple tables and creating relationships between them to achieve consistency.



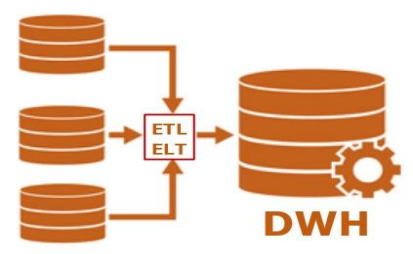
Normalization Forms

- 1NF
- 2NF
- 3NF
- BCNF



1NF

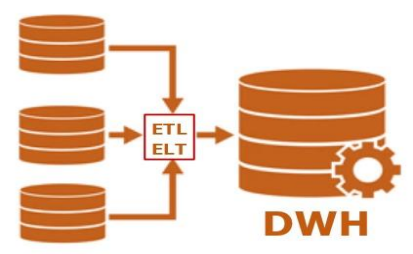
- Eliminate duplicate columns.
- Eliminate repeating groups in individual tables.
- Ensure all columns contain atomic(indivisible) values.
- Rows must be uniquely identifiable using primary key.



Un-Normalized

CustomerId	CustomerName	OrderId
10001	John Smith	SO12,SO14
10002	Dane Jackson	SO13
10003	Rakesh Kumar	SO15,SO16,S017
10004	Amanda Wilson	SO18
10005	Aman Patel	SO19,SO20

CustomerId	CustomerName	OrderId1	OrderId2	OrderId3
10001	John Smith	SO12	SO14	Null
10002	Dane Jackson	SO13	Null	Null
10003	Rakesh Kumar	SO15	SO16	S017
10004	Amanda Wilson	SO18	Null	Null
10005	Aman Patel	SO19	SO20	Null



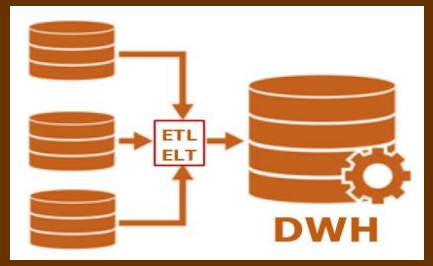
1NF

Customers

CustomerId	CustomerName
10001	John Smith
10002	Dane Jackson
10003	Rakesh Kumar
10004	Amanda Wilson
10005	Aman Patel

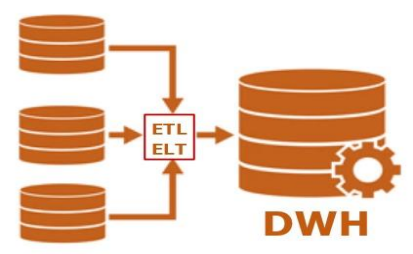
Orders

OrderId	CustomerId
SO12	**10001**
SO14	**10001**
SO13	**10002**
SO15	**10003**
SO16	**10003**
S017	**10003**
SO18	**10004**
SO19	**10005**
SO20	**10005**



2NF

- Table should be in 1NF.
- Move the columns that are not depended on primary key to a new table.



2NF

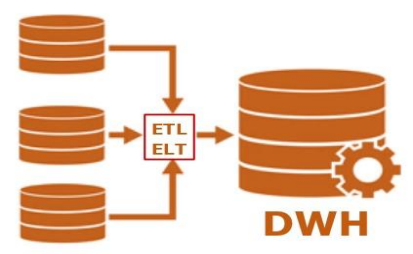
CustomerId	CustomerName	Phone	OrderId	Amount	OrderDate
10001	John Smith	1922617636	SO12	1070	2025-01-01
10001	John Smith	1922617636	SO14	1082	2025-01-02
10002	Dane Jackson	1733938383	SO13	3351	2025-01-03
10003	Rakesh Kumar	9827373713	SO15	2387	2025-01-04
10003	Rakesh Kumar	9827373713	SO16	1830	2025-01-05
10003	Rakesh Kumar	9827373713	S017	2270	2025-01-06
10004	Amanda Wilson	1838373729	SO18	3393	2025-01-07
10005	Aman Patel	8373737163	SO19	1822	2025-01-08
10005	Aman Patel	8373737163	SO20	2020	2025-01-09

Customers

CustomerId(PK)	CustomerName	Phone
10001	John Smith	1922617636
10002	Dane Jackson	1733938383
10003	Rakesh Kumar	9827373713
10004	Amanda Wilson	1838373729
10005	Aman Patel	8373737163

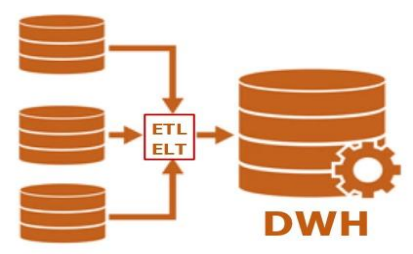
Orders

OrderId(PK)	CustomerId(FK)	Amount	OrderDate
SO12	**10001**	1070	2025-01-01
SO14	**10001**	1082	2025-01-02
SO13	**10002**	3351	2025-01-03
SO15	**10003**	2387	2025-01-04
SO16	**10003**	1830	2025-01-05
S017	**10003**	2270	2025-01-06
SO18	**10004**	3393	2025-01-07
SO19	**10005**	1822	2025-01-08
SO20	**10005**	2020	2025-01-09



3NF

- Table should be in 2NF.
- No transitive dependencies.



3NF

CustomerId(PK)	CustomerName	ProductId	ProductName
10001	John Smith	1	Laptop
10002	Dane Jackson	2	Phone
10003	Rakesh Kumar	1	Laptop
10004	Amanda Wilson	3	Headphone
10005	Aman Patel	1	Laptop

Customers

CustomerId(PK)	CustomerName	ProductId(FK)
10001	John Smith	1
10002	Dane Jackson	2
10003	Rakesh Kumar	1
10004	Amanda Wilson	3
10005	Aman Patel	1

Products

ProductId(PK)	ProductName
1	Laptop
2	Phone
3	Headphone